

OPERATING SYSTEM I (COM 311)

General Objectives

On completion of this course the student should be able to:

1. Understand Operating Systems.
2. Know the structure, functions, and philosophy of Operating Systems.
3. Understand Inter process Communication.
4. Know various Scheduling Techniques.
5. Understand Interrupt and Masking Traps.
6. Understand Operating System Kernel.
7. Know the different Operating System commands.

Unit One: MEANING OF OPERATING SYSTEMS

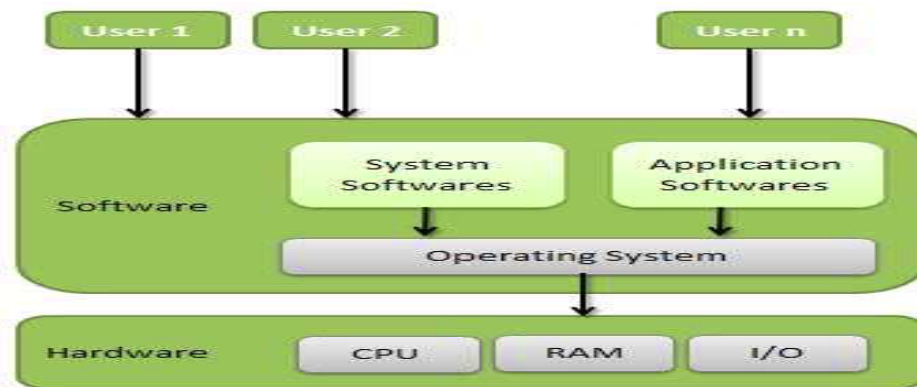
This unit gives a basic idea about Operating System starting with definition, structure and its functions.

An operating System (OS) is an intermediary between users and computer hardware. It provides users an environment in which a user can execute programs conveniently and efficiently.

In technical terms, it is software which manages hardware. An operating System controls the allocation of resources and services such as memory, processors, devices and information.

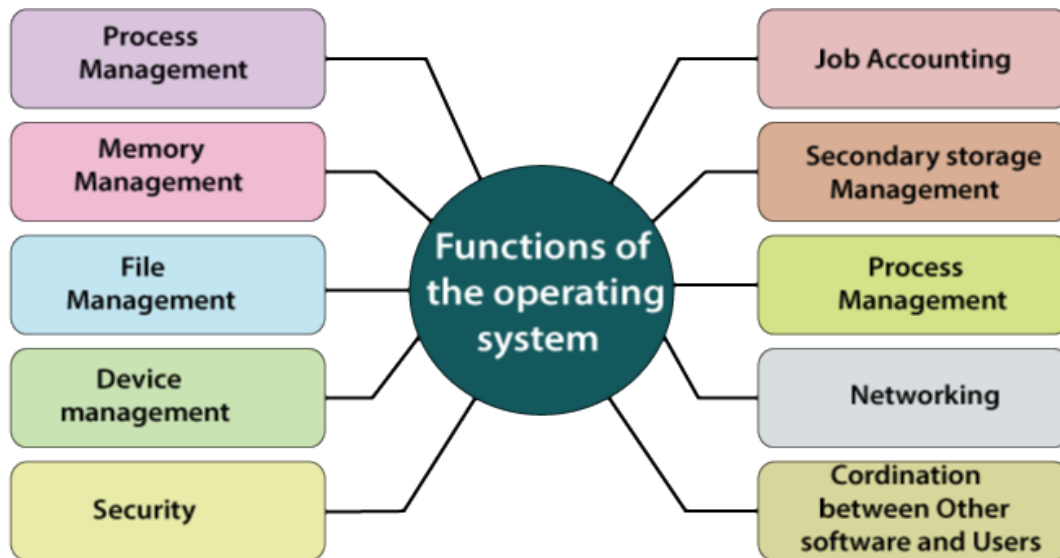
Definition

An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.



Following are some of important functions of an operating System.

1. Memory Management
2. Processor Management
3. Device Management
4. File Management
5. Coordination between other software and users
6. Control over system performance
7. Job accounting
8. Error detecting aids
9. Security



Memory Management

Memory management refers to management of Primary Memory or Main Memory. Main memory provides a fast storage that can be access directly by the CPU. So for a program to be executed, **it must be in the main memory**. Operating System does the following activities for memory management.

1. Keeps tracks of primary memory i.e. what part of it are in use by whom, what part are not in use. In multiprogramming, **OS decides which process will get memory when and how much**.
2. Allocates the memory when the process requests it to do so.
3. De-allocates the memory when the process no longer needs it or has been terminated.

Processor Management

In multiprogramming environment, **OS decides which process gets the processor when and how much time**. This function is called **process scheduling**. Operating System

does the following activities for processor management.

1. Keeps tracks of processor and status of process. Program responsible for this task is known as traffic controller.
2. Allocates the processor (CPU) to a process.
3. De-allocates processor when processor is no longer required.

Device Management

OS manages device communication via their respective drivers. Operating System does the following activities for device management.

1. Keeps tracks of all devices. Program responsible for this task is known as the I/O controller.
2. Decides which process gets the device when and for how much time.
3. Allocates the device in the efficient way.
4. De-allocates devices.

File Management

A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. Operating System does the following activities for file management.

1. Keeps track of information, location, uses, status etc. The collective facilities are often known as file system.
2. Decides who gets the resources.
3. Allocates the resources.
4. De-allocates the resources.

Other Important Activities

Following are some of the important activities that Operating System does.

Security: By means of password and similar other techniques, preventing unauthorized access to programs and data.

Control over system performance: Recording delays between request for a service and response from the system.

Job accounting: Keeping track of time and resources used by various jobs and users.

Error detecting aids: Production of dumps, traces, error messages and other debugging and error detecting aids.

Coordination between other software and users: Coordination and assignment of compilers, interpreters, assemblers and other software to the various users of the computer systems.

Components of OS: OS has two parts.

(1) Kernel.

(2) Shell.

(1) **Kernel** is an active part of an OS i.e., it is the part of OS running at all times. **It is a program which can interact with the hardware.** Example: Device driver, files, system files etc.

(2) **Shell** is also called as the **command interpreter**. It is a set of programs used to interact with the application programs. It is responsible for execution of instructions given to OS (called commands).

Operating systems can be explored from **two viewpoints: the user and the system**.

User View: From the user's point view, the OS is designed for one user to monopolize its resources, to maximize the work that the user is performing and for ease of use.

System View: From the computer's point of view, an operating system is a control program that manages the execution of user programs to prevent errors and improper use of the computer. It is concerned with the operation and control of I/O devices.

Evolution of OS:

Operating systems are there from the very first computer generation. Operating systems keep evolving over the period of time. **Following are few of the important types of operating system which are most commonly used.**

1. Batch Processing Operating System:

This type of OS accepts more than one job and these jobs are batched/ grouped together according to their similar requirements. This is done by computer operator. To speed up processing, jobs with similar needs are batched together and run as a group. Thus, the programmers left their programs with the operator. The operator then sorts programs into batches with similar requirements.

The problems with Batch Systems are following.

- a. Lack of interaction between the user and job.
- b. CPU is often idle, because the speeds of the mechanical I/O devices are slower than CPU.

- c. Difficult to provide the desired priority.

2. Time-Sharing/multitasking Operating Systems

Time sharing is a technique which **enables many people, located at various terminals, to use a particular computer system at the same time.** Time-sharing or multitasking is a logical extension of multiprogramming. **Processor's time which is shared among multiple users simultaneously is termed as time-sharing.** It provides extra facilities such as:

1. Faster switching between multiple jobs to make processing faster.
2. Allows multiple users to share computer system simultaneously.
3. The users can interact with each job while it is running.

Advantages of Timesharing operating systems are following

- a. Provide advantage of quick response.
- b. Avoids duplication of software.
- c. Reduces CPU idle time.

Disadvantages of Timesharing operating systems are following.

- a. Problem of reliability.
- b. Question of security and integrity of user programs and data.
- c. Problem of data communication.

Time-sharing or multitasking is a logical extension of multiprogramming. Processor's time which is shared among multiple users simultaneously is termed as time-sharing. The main difference between Multiprogrammed Batch Systems and Time-Sharing Systems is that in case of multiprogrammed batch systems, objective is to maximize processor use, whereas in Time-Sharing Systems objective is to minimize response time.

Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response. For example, in a transaction processing, processor execute each user program in a short burst or quantum of computation. That is if n users are present, each user can get time quantum. When the user submits the command, the response time is in few seconds at most.

3. Distributed operating System

Distributed systems use multiple central processors to serve multiple real time application and multiple users. Data processing jobs are distributed among the processors accordingly to which one can perform each job most efficiently.

The processors communicate with one another through various communication lines (such as high-speed buses or telephone lines). These are referred as loosely coupled systems or distributed systems. Processors in a distributed system may vary in size and function. These processors are referred as sites, nodes, and computers and so on.

The advantages of distributed systems are following.

- a. With resource sharing facility user at one site may be able to use the resources available at another.
- b. Speedup the exchange of data with one another via electronic mail.
- c. If one site fails in a distributed system, the remaining sites can potentially continue operating.
- d. Better service to the customers.
- e. Reduction of the load on the host computer.
- f. Reduction of delays in data processing.

It can be classified into two categories:

1. Client-Server systems
2. Peer-to-Peer systems

4. Network operating System

Network Operating System runs on a server and provides server the capability to manage data, users, groups, security, applications, and other networking functions. **The primary purpose of the network operating system is to allow shared file and printer access among multiple computers in a network**, typically a local area network (LAN), a private network or to other networks. Examples of network operating systems are Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD.

The advantages of network operating systems are following.

- a. Centralized servers are highly stable.

- b. Security is server managed.
- c. Upgrades to new technologies and hardware can be easily integrated into the system.
- d. Remote access to servers is possible from different locations and types of systems.

The disadvantages of network operating systems are following.

- a. High cost of buying and running a server.
- b. Dependency on a central location for most operations.
- c. Regular maintenance and updates are required.

5. Desktop Systems/Personal Computer Systems

The PC operating system is designed for maximizing user convenience and responsiveness. This system is neither multi-user nor multitasking. These systems include PCs running Microsoft Windows and the Apple Macintosh. The MS-DOS operating system from Microsoft has been superseded by multiple flavors of Microsoft Windows and IBM has upgraded MS-DOS to the OS/2 multitasking system.

6. Real-Time Operating Systems (RTOS)

Real time system is defines as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. **Real time processing is always online whereas on line system need not be real time. The time taken by the system to respond to an input and display of required updated information is termed as response time. So in this method response time is very less as compared to the online processing.**

Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application. Real-time operating system has well-defined, fixed time constraints otherwise system will fail. For example Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, and home-appliance controllers, Air traffic control system etc.

There are two types of real-time operating systems.

1. Hard real-time systems

Hard real-time systems guarantee that critical tasks complete on time. In hard real-time systems secondary storage is limited or missing with data stored in ROM. In these systems virtual memory is almost never found.

2. Soft real-time systems

Soft real time systems are less restrictive. Critical real-time task gets priority over other tasks and retains the priority until it completes. Soft real-time systems have limited utility than hard real-time systems. For example, Multimedia, virtual reality, Advanced Scientific Projects like undersea exploration and planetary rovers etc.

7. Multiprogramming Operating System:

This type of OS is used to execute more than one job simultaneously by a single processor. It increases CPU utilization by organizing jobs so that the CPU always has one job to execute. The concept of multiprogramming is described as follows:

1. All the jobs that enter the system are stored in the job pool(in disc). The operating system loads a set of jobs from job pool into main memory and begins to execute.
2. During execution, the job may have to wait for some task, such as an I/O operation, to complete. In a multiprogramming system, the operating system simply switches to another job and executes.
3. When that job needs to wait, the CPU is switched to *another* job, and so on.
4. When the first job finishes waiting and it gets the CPU back. As long as at least one job needs to execute, the CPU is never idle.

Multiprogramming operating systems use the mechanism of job scheduling and CPU scheduling.

8. Multiprocessor Operating Systems

Multiprocessor operating systems are also known as parallel OS or tightly coupled OS. **Such operating systems have more than one processor in close communication that sharing the computer bus, the clock and sometimes memory and peripheral devices. It executes multiple jobs at same time and makes the processing faster.**

Multiprocessor systems have **three main advantages**:

1. **Increased throughput:** By increasing the number of processors, the system performs more work in less time.
2. **Economy of scale:** Multiprocessor systems can save more money than multiple single-processor systems, because they can share peripherals, mass storage, and power supplies.
3. **Increased reliability:** If one processor fails to done its task, then each of the remaining processors must pick up a share of the work of the failed processor. The failure of one processor will not halt the system, only slow it down.

The ability to continue providing service proportional to the level of surviving hardware is called **graceful degradation**. Systems designed for graceful degradation are called **fault tolerant**.

The multiprocessor operating systems are classified into **two categories**:

1. Symmetric multiprocessing system
2. Asymmetric multiprocessing system

In symmetric multiprocessing system, each processor runs an identical copy of the operating system, and these copies communicate with one another as needed.

In asymmetric multiprocessing system, a processor is called master processor that controls other processors called slave processor. Thus, it establishes master-slave relationship. The master processor schedules the jobs and manages the memory for entire system.

Unit 2: Process management

Objective

1. Describe process management
2. Explain process description (Process, States and Process Control Block (PCB))
3. Describe Process Scheduling (Types, Comparison of different scheduling policies)

Introduction

This section describes process, process states, process control block (PCB) and process scheduling

Process

A process is a **program in execution**. The execution of a process must **progress in a sequential fashion**. Definition of process is following.

A process is defined as an entity which represents the **basic unit of work to be implemented in the system**.

Components of a process are following

Object Program: Code to be executed.

Data: Data to be used for executing the program.

Resources: While executing the program, it may require some resources.

Status: Verifies the status of the process execution. A process can run to completion only when all requested resources have been allocated to the process. Two or more processes could be executing the same program, each using their own data and resources.

Program

A program by itself is not a process. It is a static entity made up of program statement while process is a dynamic entity. Program contains the instructions to be executed by

processor.

A program takes a space at single place in main memory and continues to stay there. A program does not perform any action by itself.

Difference between Process and Program

S/N	Process	Program
1.	Static object	Dynamic object
2.	A program resides in secondary storage	Process resides in main memory.
3.	Span time of a program is unlimited	Span time of a process is limited
4.	A process is an 'active' entity	Program is a ' passive ' entity.
5.	A program is an algorithm expressed in programming language	Process is expressed in assembly language or machine language.
6.	Both are same with different name i.e. when this is sleeping (not executing) it is called program	It is executing becomes process.

Process State As a process executes, it changes *state*

S/N	State and Description
1.	New : The process is being created
2.	Ready : the process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating so that they can run.
3.	Running : process instructions are being executed (i.e the process that is currently

	being executed)
4.	Waiting: the process is waiting for some event to occur (such as the completion of an I/O operation.
5.	Terminated: The process has finished execution

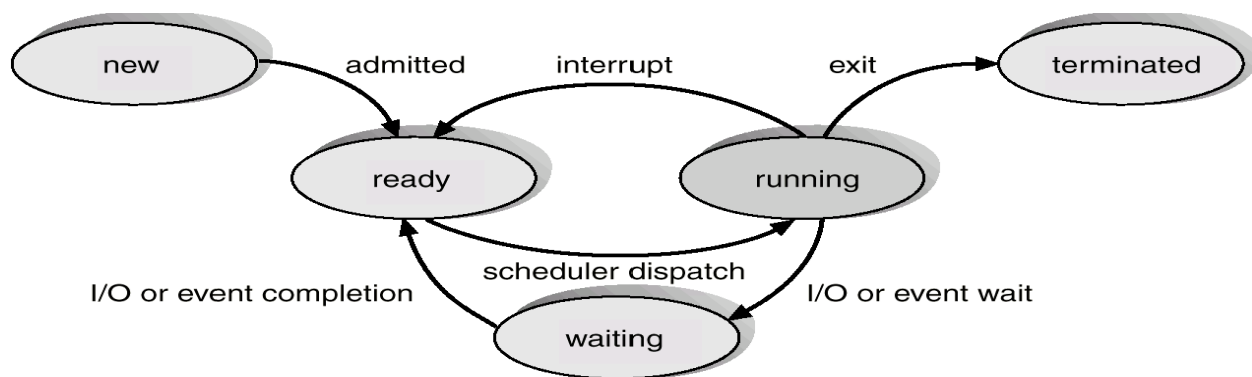


Diagram of Process State.

Process Control Block, PCB

Each process is represented in the operating system by a **process control block (PCB)** also called a **task control block**. PCB is the data structure used by the operating system. Operating system groups all information that needs about particular process.

PCB contains many pieces of information associated with a specific process which is described below.

1. **Pointer:** Points to another process control block. Pointer is used for maintaining the scheduling list.
2. **Process State** Process state may be new, ready, running, waiting and so on.
3. **Program Counter** Program Counter indicates the address of the next instruction to be executed for this process.

4. **CPU registers** CPU registers include general purpose register, stack pointers, index registers and accumulators etc. number of register and type of register totally depends upon the computer architecture.
5. **Memory management information** This information may include the value of base and limit registers, the page tables, or the segment tables depending on the memory system used by the operating system. **This information is useful for deallocating the memory when the process terminates.**
6. **Accounting information** This information includes the amount of CPU and real time used time limits, job or process numbers, account numbers etc.

The PCB simply serves as the repository for any information that may vary from process to process.

Process scheduling

The process scheduling is the activity of the process manager that handles the **removal of the running process from the CPU** and the **selection of another process** on the basis of a particular strategy.

Process scheduling is an essential part of a Multiprogramming operating system. Such operating systems allow more than one process to be loaded into the executable memory at a time and loaded process shares the CPU using time multiplexing.

Scheduling Queues

Scheduling queues refers to queues of processes or devices. When the process enters into the system, then this process is put into a job queue. This queue consists of all processes in the system. The operating system also maintains other queues such as device queue. Device queue is a queue for which multiple processes are waiting for a particular I/O device. Each device has its own device queue.

Process Scheduling Queues

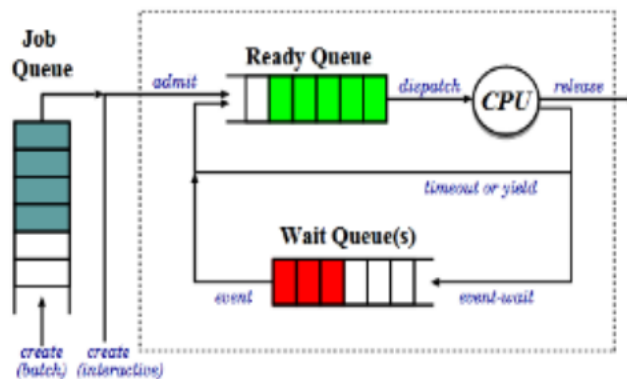
Job Queue: This queue consists of all processes in the system; those processes are entered to the system as new processes.

Ready Queue: This queue consists of the processes that are residing in **main**

memory and are ready and waiting to execute by CPU. This queue is generally stored as a linked list. A ready-queue header contains pointers to the first and final PCBs in the list. Each PCB includes a pointer field that points to the next PCB in the ready queue.

Device Queue: This queue consists of the processes that are waiting for a particular I/O device. Each device has its own device queue.

Scheduling Queues



A scheduler is a decision maker that selects the processes from one scheduling queue to another or allocates CPU for execution.

Schedulers are special system software which handles process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run.

The Operating System has **three types of scheduler**:

1. Long-term scheduler or Job scheduler
2. Short-term scheduler or CPU scheduler
3. Medium-term scheduler

Long-term scheduler or Job scheduler

It is also called job scheduler. Long term scheduler determines which programs are admitted to the system for processing. Job scheduler selects processes from the queue and loads them into memory for execution. Process loads into the memory for

CPU scheduling. The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler. When process changes the state from new to ready, then there is use of long term scheduler. In summary:

The long-term scheduler or job scheduler selects processes from discs and loads them into main memory for execution. It executes much less frequently.

It controls the degree of multiprogramming (i.e., the number of processes in memory).

Because of the longer interval between executions, the long-term scheduler can afford to take more time to select a process for execution.

Short-term scheduler or CPU scheduler

It is also called CPU scheduler. Main objective is increasing system performance in accordance with the chosen set of criteria. **It is the change of ready state to running state of the process. CPU scheduler selects process among the processes that are ready to execute and allocates CPU to one of them.** Short term scheduler also **known as dispatcher**. Short term scheduler is faster than long term scheduler. In summary:

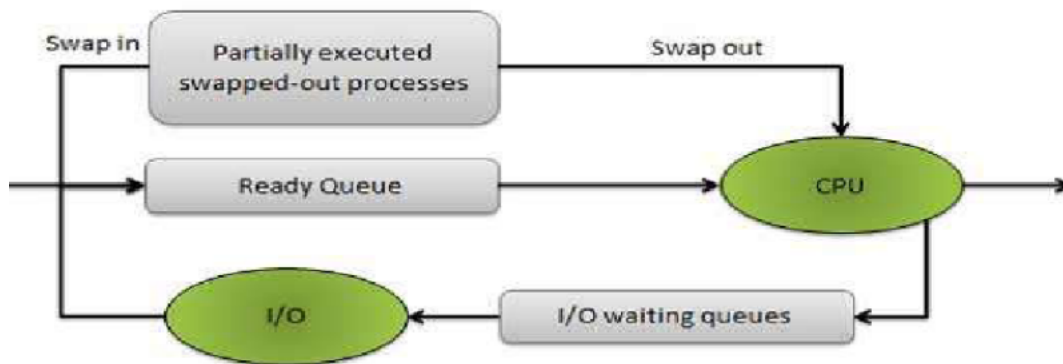
The short-term scheduler or CPU scheduler selects a process from among the processes that are ready to execute and allocates the CPU.

The short-term scheduler must select a new process for the CPU frequently. A process may execute for only a few milliseconds before waiting for an I/O request.

Medium-term scheduler

The medium-term scheduler schedules the processes as intermediate level of scheduling. Medium term scheduling is part of the swapping. It removes the processes

from the memory. It reduces the degree of multiprogramming. The medium term scheduler is in-charge of handling the swapped out-processes.



Running process may become suspended if it makes an I/O request. Suspended processes cannot make any progress towards completion. **In this condition, to remove the process from memory and make space for other process, the suspended process is moved to the secondary storage. This process is called swapping**, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

Comparison between Scheduler

S/ N	Long Term Scheduler	Short Term Scheduler	Medium Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.

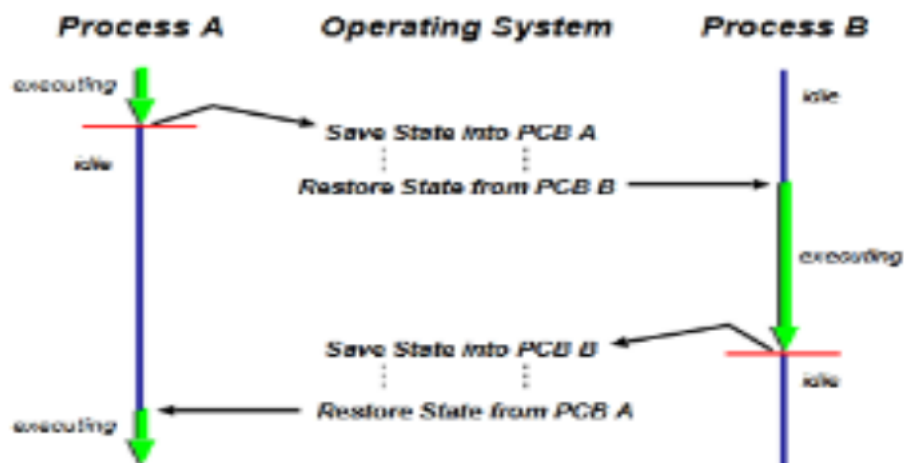
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.
---	---	---	---

Two State Process Model

Two state process models refer to running and non-running states which are described below.

- 1. Running** When new process is created by Operating System that process enters into the system in the running state.
- 2. Non-Running** Processes that are not running are kept in queue, waiting for their turn to execute. Each entry in the queue is a pointer to a particular process. Queue is implemented by using linked list. Use of dispatcher is as follows. When a process is interrupted, that process is transferred in the waiting queue. If the process has completed or aborted, the process is discarded. In either case, the dispatcher then selects a process from the queue to execute.

Context Switch



When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process.

Context-switch time is overhead; the system does no useful work while switching.

Time dependent on hardware support.

A context switch is the mechanism to store and restore the state or context of a CPU in Process Control block so that a process execution can be resumed from the same point at a later time. Using this technique a context switcher enables multiple processes to share a single CPU. Context switching is an essential part of a multitasking operating system features.

When the scheduler switches the CPU from executing one process to execute another, the context switcher saves the content of all processor registers for the process being removed from the CPU, in its process descriptor. When the process is switched, the following information is stored.

1. Program Counter
2. Scheduling Information
3. Base and limit register value
4. Currently used register
5. Changed State
6. I/O State
7. Accounting

UNIT 3: Understand Inter process Communication

Objective

1. Define process concepts
2. Explain Process creation and process termination
3. Describe Inter process communication (IPC) techniques
4. Explain process states, process table

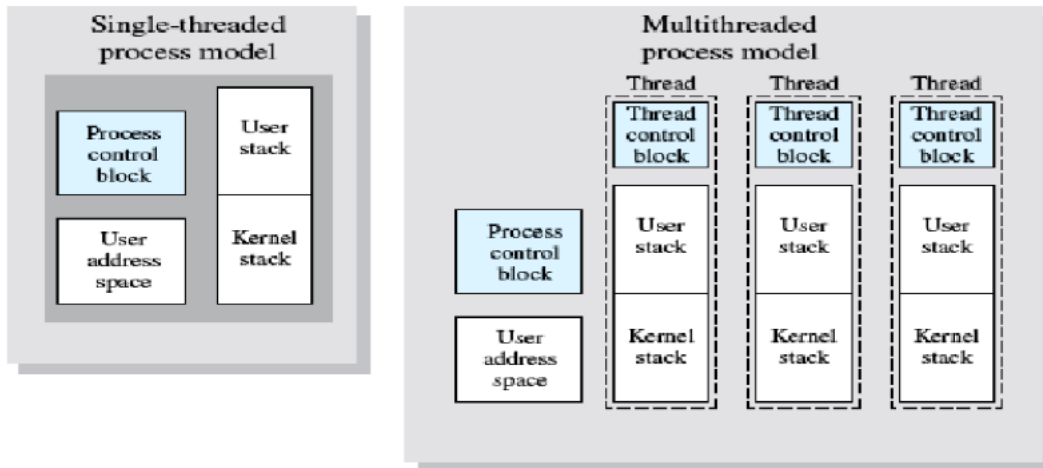
Process

A process is a program in execution. The execution of a process must progress in a sequential fashion. Definition of process is following.

A process is defined as an entity which represents the basic unit of work to be implemented in the system

Thread

A thread, sometimes called a lightweight process (LWP), is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack. We have Single-threaded and multithreaded



Ex: A web browser might have one thread display images or text while another thread retrieves data from the network. A word processor may have a thread for displaying graphics, another thread for reading keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

In certain situations a single application may be required to perform several similar tasks. For example, a web server accepts client requests for web pages, images, sound, and so forth. A busy web server may have several (perhaps hundreds) of clients concurrently accessing it. If the web server ran as a traditional single threaded process, it would be able to service only one client at a time.

One solution is to have the server run as a single process that accepts requests. When the server receives a request, it creates a separate process to service that request. In fact, this process-creation method was in common use before threads became popular. Process creation is very heavyweight, as was shown in the previous chapter. If the new process will perform the same tasks as the existing process, why incur all that overhead? It is generally more efficient for one process that contains multiple threads to serve the same purpose. This approach would multithread the web-server process. The server would create a separate thread that would listen for client requests; when a request was made, rather than creating another process, it would create another thread to service the request.

Benefits / Advantages

The benefits of multithreaded programming can be broken down into four major categories:

1. Responsiveness: Multithreading an interactive application may allow a program to continue running even if part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. For instance, a multithreaded web browser could still allow user interaction in one thread while an image is being loaded in another thread.

2. Resource sharing: By default, threads share the memory and the resources of the process to which they belong. The benefit of code sharing is that **it allows an application to have several different threads of activity all within the same address space.**

3. Economy: Allocating memory and resources for process creation is costly. Alternatively, because threads share resources of the process to which they belong, it is more economical to create and context switch threads.

4. Utilization of multiprocessor architectures: The benefits of multithreading can be greatly increased in a multiprocessor architecture, **where each thread may be running in parallel on a different processor.** A single threaded process can only run on one CPU, no matter how many are available.

Multithreading on a multi-CPU machine increases concurrency. In single processor architecture, the CPU generally moves between each thread so quickly as to create an illusion of parallelism, but in reality only one thread is running at a time.

Process vs. Thread

S/ N	Process	Thread
1	Process cannot share the same memory area (address space).	Threads can share memory and files.
2	It takes more time to create a process	It takes less time to create a

		thread.
3	It takes more time to complete the execution and terminate.	3.Less time to terminate
4	Execution is very slow.	Execution is very fast.
5	It takes more time to switch between two processes.	It takes less time to switch between two threads.
6	System calls are required to communicate each other	System calls are not required.
7	It requires more resources to execute.	Requires fewer resources.
8	Implementing the communication between processes is bit more difficult.	Communication between two threads are very easy to implement because threads share the memory

Types of Thread

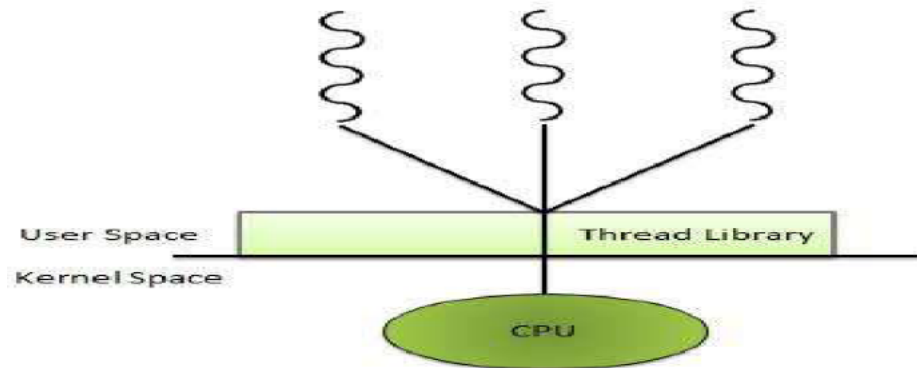
Threads are implemented in following **two ways**

1. **User Level Threads** -- User managed threads
2. **Kernel Level Threads** -- Operating System managed threads acting on kernel, an operating system core.

User Level Threads

In this case, application manages thread management kernel is not aware of the existence of threads. The thread library contains code for creating and destroying threads, for passing message and data between threads, for scheduling thread execution and for saving and restoring thread contexts. The application begins with a

single thread and begins running in that thread.



User-Level Threads

User-level threads implement in user-level libraries, rather than via systems calls, so thread switching does not need to call operating system and to cause interrupt to the kernel. In fact, the kernel knows nothing about user-level threads and manages them as if they were single-threaded processes.

Advantages:

1. User-level threads do not require modification to operating systems.
2. Simple Representation: Each thread is represented simply by a PC, registers, stack and a small control block, all stored in the user process address space.
3. Simple Management: This simply means that creating a thread, switching between threads and synchronization between threads can all be done without intervention of the kernel.
4. Fast and Efficient: Thread switching is not much more expensive than a procedure call.

Disadvantages:

1. There is a lack of coordination between threads and operating system kernel.
2. User-level threads require non-blocking systems call i.e., a multithreaded kernel.

Kernel Level Threads

In this case, thread management done by the Kernel. There is no thread management code in the application area. Kernel threads are supported directly by the operating system. Any application can be programmed to be multithreaded. All of the threads

within an application are supported within a single process.

The Kernel maintains context information for the process as a whole and for individuals' threads within the process. Scheduling by the Kernel is done on a thread basis. The Kernel performs thread creation, scheduling and management in Kernel space. Kernel threads are generally slower to create and manage than the user threads.

Advantages:

1. Because kernel has full knowledge of all threads, Scheduler may decide to give more time to a process having large number of threads than process having small number of threads.
2. Kernel-level threads are especially good for applications that frequently block.

Disadvantages:

The kernel-level threads are slow and inefficient. For instance, threads operations are hundreds of times slower than that of user-level threads. Since kernel must manage and schedule threads as well as processes. It requires a full thread control block (TCB) for each thread to maintain information about threads. As a result there is significant overhead and increased in kernel complexity.

Difference between User Level & Kernel Level Thread

S.N.	User Level Threads	Kernel Level Thread
1	User level threads are faster to create and manage.	Kernel level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User level thread is generic and can run on any operating system.	Kernel level thread is specific to the operating system.
4	Multi-threaded application cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

Multithreading Models

Many systems provide support for both user and kernel threads, resulting in different multithreading models. We look at **three common types of threading implementation**.

Many-to-One Model

The many-to-one model maps many user-level threads to one kernel thread. Thread management is done in user space, so it is efficient, but the entire process will block if a thread makes a blocking system call.

Also, because only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors.

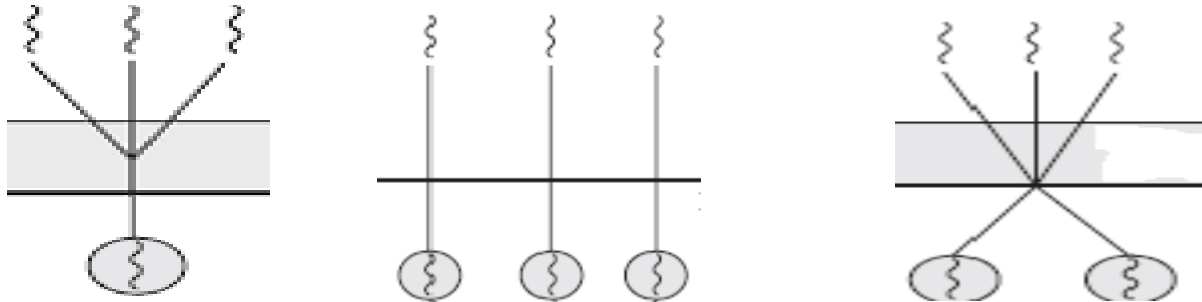
One-to-one Model

The one-to-one model maps each user thread to a kernel thread. It provides more concurrency than the many-to-one model by allowing another thread to run when a thread makes a blocking system call; **it also allows multiple threads to run in parallel on multiprocessors.** The **only drawback to this model is that creating a user thread requires creating the corresponding kernel thread.** Because the overhead of creating kernel threads can burden the performance of an application, most implementations of this model restrict the number of threads supported by the system. Windows NT, Windows 2000, and OS/2 implement the one-to-one model.

Many-to-Many Model

The many-to-many model multiplexes many user-level threads to a smaller or equal number of kernel threads. The number of kernel threads may be specific to either a particular application or a particular machine (an application may be allocated more kernel threads on a multiprocessor than on a uniprocessor). Whereas the many-to-one model allows the developer to create as many user threads as she wishes, true concurrency is not gained because the kernel can schedule only one thread at a time. The one-to-one model allows for greater concurrency, but the developer has to be careful not to create too many threads within an application (and in some instances

may be limited in the number of threads she can create). The many-to-many model suffers from neither of these shortcomings: Developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor.



(Diagram of many-to-one model, one-to-one model and many-to-many model)

Process Creation

Parent process creates children processes, which, in turn create other processes, forming a **tree of processes**.

Process Termination

Process executes last statement and asks the operating system to decide it (**exit**).

Cooperating Processes

The concurrent processes executing in the operating system may be either independent processes or cooperating processes. A process is **independent** if it cannot affect or be affected by the other processes executing in the system. Clearly, any process that does not share any data (temporary or persistent) with any other process is independent. On the other hand, a process is **cooperating** if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

Advantages of process cooperation

1. **Information sharing:** Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to these types of resources.
2. **Computation speedup:** If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Such a

speedup can be achieved only if the computer has multiple processing elements (such as CPUS or I/O channels).

3. **Modularity:** We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads
4. **Convenience:** Even an individual user may have many tasks on which to work at one time. For instance, a user may be editing, printing, and compiling in parallel.

Concurrent execution of cooperating processes requires mechanisms that allow processes to communicate with one another and to synchronize their actions.

Inter-process Communication (IPC)

UNIT 4: Know various Scheduling Techniques

Objective

1. Define CPU Scheduling
2. List type of scheduling
3. Explain CPU scheduling criteria: preemptive and non-preemptive
4. Describe Scheduling Algorithms (First Come First Serve (FCFS), Shortest-Job-First (SJF), Priority, Round Robin (RR), Multilevel Queue (MQ), Multilevel Feedback Queue (MFQ)
5. Recognize: Multiprogramming, Multiprocessing, Multitasking, and Multithreading

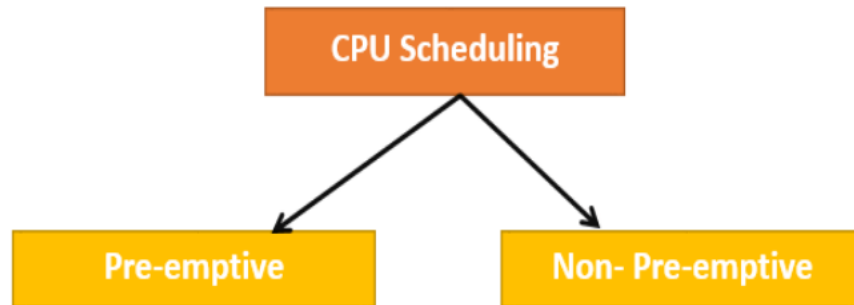
PROCESS SCHEDULING ALGORITHMS

This section describes various scheduling algorithms like FCFS, SJF, RR and Multilevel Queue Scheduling.

CPU Scheduling is a process of determining which process will own CPU for execution while another process is on hold. The main task of CPU scheduling is to make sure that whenever the CPU remains idle, the OS at least select one of the processes available in the ready queue for execution. The selection process will be carried out by the CPU scheduler. It selects one of the processes in memory that are ready for execution.

Types of CPU Scheduling

Here are two kinds of Scheduling methods:



Pre-emptive Scheduling

In Pre-emptive Scheduling, the **tasks are mostly assigned with their priorities**. Sometimes it is important to run a task with a higher priority before another lower priority task, even if the lower priority task is still running. **The lower priority task holds for some time and resumes when the higher priority task finishes its execution.**

Non-Pre-emptive Scheduling

In this type of scheduling method, the CPU has been allocated to a specific process. The process that keeps the CPU busy will release the CPU either by switching context or terminating. It is the only method that can be used for various hardware platforms. That's because it doesn't need special hardware (for example, a timer) like pre-emptive scheduling.

CPU scheduling Terminologies

Burst Time/Execution Time: It is a time required by the process to complete execution. It is also called running time.

Arrival Time: when a process enters in a ready state

Finish Time: when process complete and exit from a system

Multiprogramming: A number of programs which can be present in memory at the same time.

Jobs: It is a type of program without any kind of user interaction.

User: It is a kind of program having user interaction.

Process: It is the reference that is used for both job and user.

CPU/IO burst cycle: Characterizes process execution, which alternates between CPU and I/O activity. CPU times are usually shorter than the time of I/O.

What is Dispatcher

It is a module that provides control of the CPU to the process. The Dispatcher should be fast so that it can run on every context switch. Dispatch latency is the amount of time needed by the CPU scheduler to stop one process and start another.

Functions performed by Dispatcher:

1. Context Switching
2. Switching to user mode
3. Moving to the correct location in the newly loaded program.

Types of CPU scheduling Algorithm

There are mainly **six types of process scheduling algorithms**

1. First Come First Serve (FCFS) Scheduling
2. Shortest Remaining Time
3. Priority Scheduling
4. Round Robin(RR) Scheduling
5. Shortest-Job-First (SJF) Scheduling
6. Multilevel Queue Scheduling

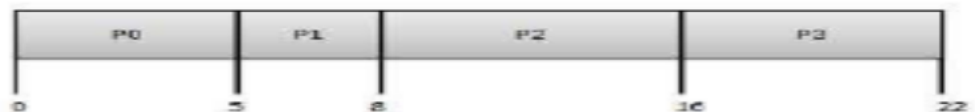


First Come First Serve

First Come First Serve is the full form of FCFS. It is the easiest and most simple CPU scheduling algorithm. In this type of algorithm, the process which requests the CPU gets the CPU allocation first. This scheduling method can be managed with a FIFO queue.

As the process enters the ready queue, its PCB (Process Control Block) is linked with the tail of the queue. So, when CPU becomes free, it should be assigned to the process at the beginning of the queue.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	5
P2	2	8	8
P3	3	6	15



Wait time of each process **First Come First Serve Scheduling Algorithm**

Process	Wait Time : Service Time - Arrival Time
P0	$0 - 0 = 0$
P1	$5 - 1 = 4$
P2	$8 - 2 = 6$
P3	$16 - 3 = 13$

Average Wait Time: $(0+4+6+13) / 4 = 5.55$

Characteristics of FCFS method:

1. It offers non-pre-emptive and pre-emptive scheduling algorithm.
2. Jobs are always executed on a first-come, first-serve basis
3. It is easy to implement and use.
4. However, this method is poor in performance, and the general wait time is quite high.

Shortest Remaining Time

The full form of SRT is Shortest remaining time. It is also known as SJF pre-emptive scheduling. In this method, the process will be allocated to the task, which is closest to its completion.

Characteristics of SRT scheduling method:

1. This method is mostly applied in batch environments where short jobs are required to be given preference.
2. This is not an ideal method to implement it in a shared system where the required CPU time is unknown.
3. Associate with each process as the length of its next CPU burst. So that operating system uses these lengths, which helps to schedule the process with the shortest possible time.

Priority Based Scheduling

Priority scheduling is a method of scheduling processes based on priority. In this method, the scheduler selects the tasks to work as per the priority.

Priority scheduling also helps OS to involve priority assignments. The processes with higher priority should be carried out first, **whereas jobs with equal priorities are carried out on a round-robin or FCFS basis.** Priority can be decided based on memory requirements, time requirements, etc.

Process	Arrival Time	Execute Time	Priority	Service Time
P0	0	5	1	0
P1	1	3	2	3
P2	2	8	1	8
P3	3	6	3	16



Wait time of each process is following

Process	Wait Time : Service Time - Arrival Time
P0	$0 - 0 = 0$
P1	$3 - 1 = 2$
P2	$8 - 2 = 6$
P3	$16 - 3 = 13$

Average Wait Time: $(0+2+6+13) / 4 = 5.25$

Round-Robin Scheduling

Round robin is the oldest, simplest scheduling algorithm. The name of this algorithm comes from the round-robin principle, where each person gets an equal share of something in turn. It is mostly used for scheduling algorithms in multitasking. This algorithm method helps for starvation free execution of processes.

Process	Arrival Time	Execute Time
P0	0	5
P1	1	3
P2	2	8
P3	3	6

Quantum = 3



Wait time of each process is following

Process	Wait Time : Service Time - Arrival Time
P0	$(0-0) + (12-3) = 9$
P1	$(3-1) = 2$
P2	$(6-2) + (15-9) = 10$
P3	$(9-3) + (18-12) = 12$

Average Wait Time: $(9+2+10+12) / 4 = 8.25$

Characteristics of Round-Robin Scheduling

1. Round robin is a hybrid model which is clock-driven
2. Time slice should be minimum, which is assigned for a specific task to be processed. However, it may vary for different processes.
3. It is a real time system which responds to the event within a specific time limit.

Shortest Job First

SJF is a full form of (Shortest job first) is a scheduling algorithm in which the **process with the shortest execution time should be selected for execution next**. This scheduling method can be pre-emptive or non-pre-emptive. It significantly reduces the average waiting time for other processes awaiting execution.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	0
P1	1	3	3
P2	2	8	8
P3	3	0	20



Wait time of each process is following

Process	Wait Time : Service Time - Arrival Time
P0	$3 - 0 = 3$
P1	$0 - 0 = 0$
P2	$16 - 2 = 14$
P3	$8 - 3 = 5$

Average Wait Time: $(3+0+14+5) / 4 = 5.50$

Characteristics of SJF Scheduling

1. It is associated with each job as a unit of time to complete.
2. In this method, when the CPU is available, the next process or job with the shortest completion time will be executed first.
3. It is implemented with non-pre-emptive policy.
4. This algorithm method is useful for batch-type processing, where waiting for jobs to complete is not critical.
5. It improves job output by offering shorter jobs, which should be executed first, which mostly have a shorter turnaround time.

Multiple-Level Queues Scheduling

This algorithm separates the ready queue into various separate queues. In this method, processes are assigned to a queue based on a specific property of the process, like the process priority, size of the memory, etc.

However, this is not an independent scheduling OS algorithm as it needs to use other types of algorithms in order to schedule the jobs.

For example, CPU-bound jobs can be scheduled in one queue and all I/O-bound jobs in another queue. The Process Scheduler then alternately selects jobs from each queue and assigns them to the CPU based on the algorithm assigned to the queue.

Characteristic of Multiple-Level Queues Scheduling:

Multiple queues should be maintained for processes with some characteristics.

1. Every queue may have its separate scheduling algorithms.
2. Priorities are given for each queue.

The Purpose of a Scheduling algorithm

Here are the reasons for using a scheduling algorithm:

1. The CPU uses scheduling to improve its efficiency.
2. It helps you to allocate resources among competing processes.
3. The maximum utilization of CPU can be obtained with multi-programming.
4. The processes which are to be executed are in ready queue.

Interrupts are signals sent to the CPU by external devices, normally I/O devices. They tell the CPU to stop its current activities and execute the appropriate part of the operating system. **Definition:** It is referred to as an input signal that has the highest priority for hardware or software events that requires immediate processing of an event. During the early days of computing, the processor had to wait for the signal to process any events. The processor should check every hardware and software program to understand if there is any signal to be processed. This method would consume a

number of clock cycles and makes the processor busy. Just in case, if any signal was generated, the processor would again take some time to process the event, leading to poor system performance.

Interrupts are important because they give the user better control over the computer. Without interrupts, a user may have to wait for a given application to have a higher priority over the CPU to be ran. This ensures that the CPU will deal with the process immediately.

There are three types of interrupts:

Hardware Interrupts are generated by hardware devices to signal that they need some attention from the OS (An electronic signal sent from an external device or hardware to communicate with the processor indicating that it requires immediate attention). They may have just received some data (e.g., keystrokes on the keyboard or an data on the Ethernet card); or they have just completed a task which the operating system previous requested, such as transferring data between the hard drive and memory.

Hardware interrupts are classified into two types

Maskable Interrupts – Processors have to interrupt mask register that allows enabling and disabling of hardware interrupts. Every signal has a bit placed in the mask register. If this bit is set, an interrupt is enabled & disabled when a bit is not set, or vice versa. Signals that interrupt the processors through these masks are referred to as masked interrupts.

Non-maskable Interrupts (NMI) – The NMIs are the highest priority activities that need to be processed immediately and under any situation, such as a timeout signal generated from a watchdog timer.

Software Interrupts are generated by programs when they want to request a [system call](#) to be performed by the operating system.

Traps are generated by the CPU itself to indicate that some error or condition

occurred for which assistance from the operating system is needed.

Polling vs Interrupts I/O

A computer must have a way of detecting the arrival of any type of input. There are two ways that this can happen, known as **polling** and **interrupts**. Both of these techniques allow the processor to deal with events that can happen at any time and that are not related to the process it is currently running.

Polling I/O

Polling is the simplest way for an I/O device to communicate with the processor. The process of periodically checking status of the device to see if it is time for the next I/O operation is called polling. The I/O device simply puts the information in a Status register, and the processor must come and get the information.

Most of the time, devices will not require attention and when one does it will have to wait until it is next interrogated by the polling program. This is an inefficient method and much of the processors time is wasted on unnecessary polls.

Compare this method to a teacher continually asking every student in a class, one after another, if they need help. Obviously the more efficient method would be for a student to inform the teacher whenever they require assistance.

Interrupts I/O

An alternative scheme for dealing with I/O is the interrupt-driven method. An interrupt is a signal to the microprocessor from a device that requires attention.

A device controller puts an interrupt signal on the bus when it needs CPU's attention when CPU receives an interrupt, It saves its current state and invokes the appropriate interrupt handler using the interrupt vector (addresses of OS routines to handle various events). When the interrupting device has been dealt with, the CPU continues with its original task as if it had never been interrupted.